

Data Handling: Import, Cleaning and Visualisation

Exercise/Workshop 2: Functions, iterations and data structures

Dr. Aurélien Sallin (Lecturer)

Ana Armendariz (Tutor)

1 October 2026

Table of contents

Before we start	1
Exercise 1: warm-up on vectors	2
Exercise 2: warm-up on for-loops	2
Exercise 3: write a sum function	2
Exercise 4: robustness and warnings	3
Exercise 5: apply the function to a matrix	3
Exercise 6: slow and fast sloths	4
Exercise 7: data structures	4
Vectors, factors and lists	4
Matrices and arrays	5
Data frames and tibbles	5
Additional exercises for training	5
Exercise 8: standard deviation	5
Exercise 9: standard error	5
Exercise 10: t-test	6
Exercise 11: data frames, tibbles and data.tables	6
All course material: https://github.com/ASallin/BEcon_3230_Datahandling_ImportCleaningandVisualization	

Before we start

Write your code in an R script saved in `2026_Data_Handling/exercises/code`, for example `exercise_02.R`. Start from the code shown in the lecture slides whenever you can. Type the code rather than copying it, and read every error message. If something does not work, raise your hand.

Exercise	What we do
1	Warm-up: vectors
2	Warm-up: for loops
3	Write a sum function
4	Make the function robust

Exercise	What we do
5	Apply the function to a matrix: <code>apply()</code> , <code>loop</code> , <code>map()</code>
6	Slow and fast sloths: a <code>while</code> loop
7	Data structures: vectors, factors, lists, matrices, arrays, data frames
Training	Standard deviation, standard error, t-test, comparing data frames, tibbles and data.tables

Exercise 1: warm-up on vectors

In the lecture we saw that atomic vectors are the most basic data structure in R, and that most operators are vectorized.

1. Create an **integer vector** `my_vector` with the values 1, 2, 3, 4, 5.
2. Add a last element, 9L, to `my_vector`.

```
my_vector <- c(my_vector, 9L)
```

3. How many elements does `my_vector` have now?
4. Extract the third element of `my_vector`.
5. Check whether 9 is an element of `my_vector`. Then check whether 10 is.

Exercise 2: warm-up on for-loops

In the lecture you saw the `for` loop: `for (i in 1:n) { ... }` repeats the body once for every value of `i`.

1. Given a vector of color names, `colors <- c("red", "green", "blue")`, write a `for` loop that prints each color name on a new line.

```
colors <- c("red", "green", "blue")
```

2. Using the same vector, write a `for` loop that checks, for each color, whether it matches your favorite color. Store your favorite color in a variable first. The loop should print `TRUE` if the color matches, and `FALSE` otherwise.

Exercise 3: write a sum function

1. Use a `for` loop and the empty function below to write a function that takes a numeric vector as input and returns the sum of its elements. Start with a total of zero and add one element per iteration.

```
my_sum <- function(x) {  
    
}
```

2. Define the vector `numbers <- c(1, 2, 3)`. Compare your function with R's built-in `sum()`: do `my_sum(numbers)` and `sum(numbers)` give the same result?

Exercise 4: robustness and warnings

1. Call your function with the following vector as input. What happens?

```
numbers2 <- c("1", "2", "3")
```

2. Why do you get an error? Compare `numbers` from Exercise 3 with `numbers2`. The functions `str()` and `class()` help.
3. Extend `my_sum()` with a control statement that checks whether the input is numeric (hint: `is.numeric(x)`). If the input is wrong, the function should issue a warning instead of stopping with an error. Use this message:

```
warning("Wrong input! This function only accepts numeric or integer values!")
```

What should the function return in that case? Think about why returning 0 would be a bad idea.

4. Test the improved function with `numbers` and `numbers2`.

Exercise 5: apply the function to a matrix

You can apply a function to every column of a matrix in several ways. Here you do the same task three times, with a loop, with `apply()` and with `map()`.

1. Create a matrix of numbers:

```
number_matrix <- matrix(1:9, nrow = 3)
```

Access the first column with `number_matrix[, 1]`.

2. Use a `for` loop to apply `my_sum()` to each column. Create an empty vector `sums_loop` first, then fill it inside the loop. Print the result.
3. Do the same with `apply()` (hint: `MARGIN = 2` applies a function over columns, as you saw in the lecture). Store the result in `sums_apply` and print it.
4. Do the same with `map_dbl()` from the `purrr` package: apply `my_sum()` to every column index of the matrix. Load the package first. Store the result in `sums_map` and print it.
5. All three give the same result. When would you use which?
6. **(Self-study)** `map_dbl()` is one member of the `map()` family. Redo point 4, but with `map()` instead of `map_dbl()`, and store the result in `sums_map2`. What changes compared to `sums_map`? Why? Use `?map` (or `help(map)`) to find out.

Exercise 6: slow and fast sloths

Loops can simulate a process over time. Write a program that calculates the populations of two kinds of sloths.

At the beginning of year 1 there are **1000 slow sloths** and **1 fast sloth**. The fast sloth is a new mutation that can use roller blades, which helps it escape from predators. Each year, every sloth has one offspring. There are no further mutations: slow sloths beget slow sloths and fast sloths beget fast sloths. Each year, 40% of all slow sloths die, but only 30% of the fast sloths.

So at the beginning of year 1 there are 1000 slow sloths. Another 1000 are born. Then 40% of these 2000 die, which leaves 1200 at the beginning of year 2. In the same year we start with 1 fast sloth, 1 more is born, and 30% of these 2 die, which leaves 1.4.

Beginning of year	Slow sloths	Fast sloths
1	1000	1
2	1200	1.4
3	1440	1.96

1. Write a **while** loop that updates both populations year by year for as long as the slow sloths are at least as numerous as the fast ones. Keep a counter for the year.
2. **In which year do the fast sloths outnumber the slow sloths for the first time?** Print the answer with `cat()`.
3. Why is a **while** loop the right tool here, and not a **for** loop?

Exercise 7: data structures

Note

This exercise also covers material from Lecture 4 (factors, lists, arrays, data frames and tibbles). Because exercise sessions are biweekly while lectures are weekly, this session falls between Lecture 3 and Lecture 4, so we work through Lecture 4 material before it is taught in class. Use this exercise as a first hands-on introduction, and revisit it after Lecture 4 if anything is still unclear.

Vectors, factors and lists

1. Create a vector `student_ages` with the ages of 5 students: 18, 21, 20, 22, 19. Print its length and its type with `length()` and `typeof()`.
2. Create a factor `student_grades` with the grades of the same 5 students: "A", "B", "A", "C", "B". Print its levels. Why would you store grades as a factor rather than as text?
3. Create a vector `student_names` with 5 names. Then create a list `student_info` that combines `student_names`, `student_ages` and `student_grades`. Access its elements by position (`[[1]]`) and by name (`$names`).
4. What is the difference between a vector and a factor? Why do you use a list, and not a vector, to store data of different types together?

Matrices and arrays

1. Create a matrix `scores_matrix` with the scores of 3 students in 3 subjects. Give it row and column names with `rownames()` and `colnames()`. Print the matrix and the score of one student in one subject.
2. Create a 3-dimensional array `student_array` with the same students and subjects, but for two semesters with different scores. Access the scores of “Student 3” in the first subject across both semesters.
3. What is the difference between a matrix and an array? When would you use an array?

Data frames and tibbles

1. Create a data frame `students_df` that combines `student_names`, `student_ages`, `student_grades` and a new vector `attendance` with the class attendance in percent: 85, 92, 78, 88, 90. Access the `attendance` column, then the row of the third student.
2. Why is a data frame better suited to this kind of data than a matrix? And what is the difference between a data frame and a list?
3. Convert `students_df` to a tibble with `as_tibble()` from the `tibble` package. Call it `students_tibble`. Print both and compare the output.

Additional exercises for training

These go beyond what we cover in the session. They are good practice for the exam.

Exercise 8: standard deviation

1. In the lecture you implemented a function that computes the mean. Now write a function `my_sd()` that computes the standard deviation,

$$SD = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2}.$$

Most R operators work element by element on a whole vector, so you do not need a loop.

2. Test it against R's `sd()`.

Exercise 9: standard error

Using `my_sd()`, write a function `my_se()` that computes the standard error of the sample mean,

$$SE_{\bar{x}} = \frac{SD}{\sqrt{n}}.$$

Exercises 8 and 9 show how to approach a bigger problem by breaking it into smaller ones: computing SD is one part of computing $SE_{\bar{x}}$, and you solved it separately.

Exercise 10: t-test

Using `my_se()`, write a function `my_t_test(x, mu_0)` for the one-sample [t-test](#),

$$t = \frac{\bar{x} - \mu_0}{SE_{\bar{x}}}.$$

Compare its output with `t.test(x, mu = mu_0)$statistic`.

Exercise 11: data frames, tibbles and data.tables

1. Convert `students_df` to a `data.table` with `as.data.table()` from the `data.table` package. Call it `students_dt`. Select the column `ages`, then filter the rows where the grade is “A”.
2. Do the following tasks three times, once with the data frame, once with the tibble (using `dplyr`) and once with the `data.table`, and compare the syntax:
 - keep the students with attendance above 85
 - compute the mean age of the students with grade “A” or “B”
 - add a column `grade_numeric` with 4 for “A”, 3 for “B” and 2 for “C”